

Content-Defined Chunking with tight chunk size bounds

Ed Schouten
ed@nuxi.nl

August 2026

Abstract

Content-Defined Chunking (CDC) algorithms cut the contents of large files into small chunks whose sizes lie within a configured range. The points at which cuts are made are intended to remain as stable as possible, even if parts of data are added, modified, or removed. This makes these algorithms a building block for storage systems that support fine-grained deduplication.

Existing CDC algorithms tend to create chunks that still have a large spread in size (e.g., a factor 32 for FastCDC). In this paper we present novel algorithms named RecMaxCDC and RepMaxCDC that yield chunks with sizes in range $[n, 2n)$. Despite the tight bound, benchmarks show an improvement in deduplication rates. In addition to that, the novel algorithms are easier to tune and parallelize.

1 Introduction

Many cloud based services depend on object storage. For example, an email service may store files that are attached to messages as objects in an object store. A video streaming service may use it to store copies of videos. A distributed software build service may use it to store input files (source code) and output files (object files, executables, execution logs).

Objects in such data stores are identified by a key which clients may provide at a later point in time to reobtain its contents. Many object stores allow clients to freely select the key to be associated with the object at the time of uploading. Such systems are called *key-value stores*.

There also exists a special type of object store where neither the client nor the server can freely select an object's key. Instead, the key is based on properties that are derived from the object's contents,

such as a cryptographic hash, the object's size, its type, or a combination thereof. These systems are referred to as Content-Addressable Storage (CAS) systems.

As CAS systems are incapable of storing the same object under multiple keys, they may deduplicate data even in scenarios where it is otherwise hard to trace objects as they pass through the system. For example, an email service that stores attachments in a CAS will be able to deduplicate files across all users' mailboxes, even if messages escape and re-enter the system by being forwarded to and from external email providers.

For an email service such a solution alone may already provide acceptable amounts of deduplication. However, for distributed software builds we observe that this is not the case. Slight alterations to source files necessitate the regeneration of object files. Object files are then linked to construct substantially larger executables. Executables are bundled into large archives, container images, and installation media. A change to a single line of code may thus result in newly observed assets having a combined size in the order of gigabytes. For such workloads it is crucial that deduplication isn't merely performed on a per-file basis, but also against parts of files.

One way to achieve fine-grained deduplication is to split files into chunks prior to storing them in a CAS. For this it is interesting to use Content-Defined Chunking (CDC) functions. These functions attempt to select cutting points that remain stable, even when files mutate.

In Section 2 we start off with discussing the overall design of the most commonly used CDC functions, which Gregoriadis *et al.* refer to as Basic Sliding Window (BSW) CDC functions. [5] There we highlight some of this design's weaknesses.

In the sections that follow we go on a journey to de-

sign a new CDC function that addresses these weaknesses. First we present MaxCDC in Section 3, which we use to explore to what extent we can address them by making incremental design changes to traditional BSW CDC functions. In Section 4 we describe a novel algorithm named RecMaxCDC that differs from existing designs by using a divide-and-conquer approach. As existing applications of CDC functions often depend on creating chunks in a streaming manner using a fixed-size read buffer, we use Section 5 to present a slightly altered version of RecMaxCDC named RepMaxCDC that facilitates this. This algorithm is intended to be a drop-in replacement for existing CDC functions.

We conclude this paper by performing some benchmarks in Section 6. These benchmarks have multiple purposes. First, they are used to determine how efficient the novel algorithms are at deduplicating data. Second, we use them to check to what extent predictions of the algorithms’ behavior hold up when using realistic data sets. Finally, they provide insight in how to configure and tune these algorithms properly.

2 Basic Sliding Window CDC functions

Many commonly used CDC functions share the same fundamental design. Namely, they perform a forward scan through an input file, computing hashes over a fixed-size sliding window. At points where the hash value matches a certain pattern, a cut is placed. Algorithm 1 shows what a simple implementation may look like.

For readability, *Hash* is modeled in Algorithm 1 as a regular hash function that is invoked repeatedly against inputs of a fixed *WindowSize*. However, practical implementations will use a rolling hash function. Rolling hash functions allow the hashing state to be reused across iterations, preventing individual bytes of data from being hashed during each of the *WindowSize* iterations that they are part of the sliding window.

Commonly used hash functions are Rabin fingerprinting [8], Buzhash [3], and Gear [14]. There are also more elaborate constructions where cryptographic block ciphers are used, resulting in a Keyed Content-Defined Chunking (KCDC) scheme. Such schemes are often used by secure backup services to protect against fingerprinting attacks. [10]

The number of 1-bits in *Mask* controls the expected

Algorithm 1 BSW chunking

```

procedure GETFIRSTCHUNKLENGTH(data)
   $n \leftarrow |data|$ 
  if  $n \leq WindowSize$  then
    // File is too small to hash.
    return  $n$ 
  end if

  for  $i \leftarrow WindowSize$  to  $n$  do
     $h \leftarrow Hash(data[i - WindowSize : i])$ 
    if  $h \& Mask = 0$  then
      // Found a cutting point.
      return  $i$ 
    end if
  end for
  // No cutting point found. Cut at end-of-file.
  return  $n$ 
end procedure

```

distance from *WindowSize* at which cuts are made. For random inputs and an unbiased hash function, the expected chunk size is $WindowSize - 1 + 2^b$, where b the number of 1-bits. The distribution of chunk sizes is geometric.

2.1 Bounded chunk sizes

There is often a desire to bound the sizes of chunks generated by CDC functions. Having a lower bound ensures that the fixed cost of RPCs and storage operations don’t dominate. It may also increase throughput of the CDC function, as the region before the lower bound does not need to be hashed. Xia *et al.* refer to this optimization as *sub-minimum chunk cut-point skipping*. [15] Having an upper bound ensures chunks can be transmitted and stored by systems that place hard limits on object sizes.

Algorithm 2 shows what a BSW CDC function that enforces both a minimum and maximum chunk size may look like. As can be seen, it creates chunks of the maximum size in case no cutting point is found within the permitted range, which effectively means it falls back to creating fixed-size chunks. Because this leads to worse deduplication, this should be avoided as much as possible.

For random inputs, the probability of reaching the maximum chunk size are low. For example, given a *Mask* having eleven 1-bits and $MaxSize = MinSize + 16384$, this only has a $\frac{2047}{2048}^{16384} \approx 0.0335\%$ chance of occurring. However, for realistic data sets this proba-

Algorithm 2 Bounded BSW chunking

```
procedure GETFIRSTCHUNKLENGTH(data)  
  // Limit the amount of lookahead.  
   $n \leftarrow \min(|data|, MaxSize)$   
  if  $n \leq MinSize$  then  
    // File is too small to chunk.  
    return  $n$   
  end if  
  
  for  $i \leftarrow MinSize$  to  $n$  do  
     $h \leftarrow Hash(data[i - WindowSize : i])$   
    if  $h \& Mask = 0$  then  
      // Found a cutting point.  
      return  $i$   
    end if  
  end for  
  // Forced cut at the maximum size.  
  return  $n$   
end procedure
```

bility is significantly higher due to repetition of data. This is why over time several extensions have been published.

The Two Thresholds, Two Divisors (TTTD) algorithm [4] scans the input data using two masks: *Mask1* and *Mask2*. If no cutting point is found using *Mask1*, another scan of the same region is performed using *Mask2*. By letting the 1-bits in *Mask2* be a strict subset of those in *Mask1*, the second pass is capable of selecting cutting points more liberally.

FastCDC [15] attempts to address this problem differently by reducing the number of 1-bits in *Mask* if the size of the scanned region starts to exceed a threshold. This causes matching in the latter part to become less picky. This results in a chunk size distribution that is the concatenation of two geometric distributions. As this somewhat resembles a normal distribution, this approach is referred to as *Normalized Chunking (NC)*.

A downside of these extensions is that they make the algorithm more complex to configure. TTTD has four configurable parameters: *MinSize*, *MaxSize*, *Mask1*, and *Mask2*. FastCDC has one more: *MinSize*, *NormalSize*, *MaxSize*, *MaskS*, and *MaskL*. Selecting their values optimally is not intuitive, and requires intricate knowledge of the expected input data.

Furthermore, even with these extensions in place the spread in chunk sizes remains relatively high. For example, Xia *et al.* observed that for FastCDC *MaxSize* needs to be set as high as $32 \cdot MinSize$ to reach

a 0.03% probability of not finding a suitable cutting point before reaching *MaxSize*. [15]

2.2 Parallelism

When processing large input files, it may be desirable to speed up chunking by using multi-threading. Though it is possible to simply partition a file into equally sized parts and apply the BSW CDC function against each part, this wouldn't necessarily produce the same cutting points near the partition boundaries as the algorithm traditionally would. This is referred to as violating *chunking invariability*. [12]

P-Dedupe [13] solves this by having a postprocessing step that re-chunks the partition boundaries to stitch the results together correctly. P-Dedupe only applies this re-chunking process against the pairs of chunks adjacent to the partition boundaries, which is not sufficient when using a CDC algorithm that uses bounded chunk sizes. In that case there is no limit to the number of chunks following a boundary that need to be readjusted.

SS-CDC [6] addresses this by splitting up the CDC function into two separate stages. In the first stage it extracts a list of all points in the file having a rolling hash value matching *Mask*, storing them in a bit array. This stage may easily be parallelized. The second stage that selects cutting points from the bit array, respecting the chunk size bounds, runs sequentially.

A downside of SS-CDC's approach is that it prevents any follow-up steps (e.g., hashing chunks and writing them to storage) from taking place until chunking of previous partitions completes. Storing all potential cutting points in a bit array may also consume a significant amount of memory. MUCH [12] alleviates this by using a CDC function that is capable of reporting the first chunk in a partition that can no longer be affected by any data in the previous partition, which is referred to as the *marker chunk*. This at least allows the majority of chunks to be processed immediately.

An overall limitation of most of these approaches is that they assume that the underlying CDC function always selects cutting points using a constant value for *Mask*, which makes them incompatible with TTTD and FastCDC's Normalized Chunking. Multi-threading is therefore rarely used in practice.

It should be noted that there are also efforts to speed up CDC functions using hardware instructions level parallelism (SIMD). [6] [11] These will not be discussed in detail in this paper, as in the context

of BSW CDC functions they mostly pertain to the design of rolling hash functions – not the high-level structure of the CDC function itself.

3 MaxCDC: maximize the rolling hash

As discussed in Section 2.1, the bounds on the sizes of chunks generated by BSW CDC functions are by no means tight. What prevents these functions from achieving tight bounds is that they search for points in the input matching a fixed criterium. As such points only occur at a low density, this approach will always run at risk of not finding any suitable points, forcing the algorithm to cut at the maximum chunk size.

To address this, we’re going to design a new CDC function that works with a different assumption, namely that any point in a file is a valid cutting point. It’s just that some points are more desirable than others. The goal of the algorithm is to select the cutting points that are the most desirable, while respecting the minimum and maximum chunk size limits. For this we may simply search for the point at which the rolling hash value is maximal, as shown in Algorithm 3.

Algorithm 3 MaxCDC

```

procedure GETFIRSTCHUNKLENGTH(data)
  // Limit the amount of lookahead.
   $n \leftarrow \min(|data|, MaxSize + MinSize)$ 
  if  $n < 2 \cdot MinSize$  then
    // File is too small to chunk.
    return  $n$ 
  end if

   $i^* \leftarrow MinSize$ 
   $h^* \leftarrow Hash(data[i^* - WindowSize : i^*])$ 
  for  $i \leftarrow i^* + 1$  to  $n - MinSize$  do
     $h \leftarrow Hash(data[i - WindowSize : i])$ 
    if  $h^* < h$  then
      // Found a more desirable cutting point.
       $h^* \leftarrow h$ 
       $i^* \leftarrow i$ 
    end if
  end for
  // Use the most desirable cutting point.
  return  $i^*$ 
end procedure

```

3.1 Chunk size distribution

One may assume that for random inputs this algorithm yields chunks having sizes that follow a uniform distribution. However, this only holds when $r < 2$, where chunk size ratio $r = MaxSize/MinSize$. For larger chunk size ratios we observe that the chunk size distribution resembles a trapezoid, as shown in Figure 1a. This is caused by the fact that points close the minimum chunk size already survived up to $r - 1$ previous iterations due to their hash value being sufficiently low. The probability of these points having a maximal hash value is thus lower than those of points between offsets $(MaxSize - MinSize, MaxSize]$, which are only considered for the very first time.

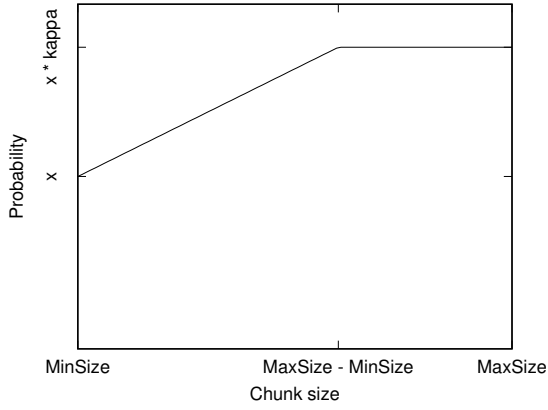
The contrast between the probabilities of creating chunks having the minimum and maximum chunk size – which we will refer to as κ – is a function of the chunk size ratio. By simulating 10^9 random chunking operations each for different chunk size ratios in $2 \leq r \leq 1000$, we were able to approximate κ . A Padé approximant of order 2 is given in Equation 1, for which a graph is provided in Figure 1b.

$$\kappa = \frac{P(X = MaxSize)}{P(X = MinSize)} \approx \begin{cases} 1 & 1 \leq r < 2 \\ \frac{-1.659 - 0.772r + 2.564r^2}{-0.888 + 1.968r + r^2} & 2 \leq r \end{cases} \quad (1)$$

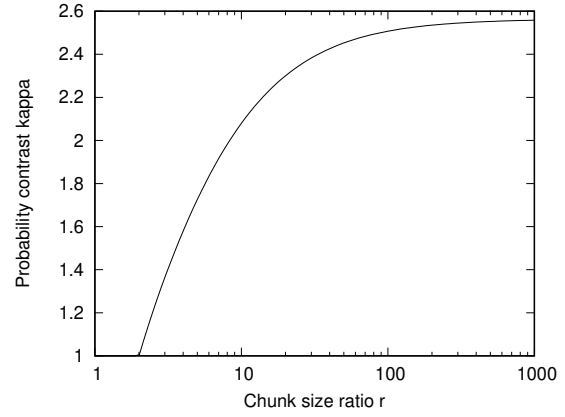
3.2 Average chunk size

The non-uniform chunk size distribution for $r \geq 2$ also causes the mean size of chunks generated by the algorithm to be skewed. Through the same simulation of random chunking operations we were able to derive an approximation of the mean chunk size μ_{out} , which is given in Equation 2. A graph of skew factor S_{out} that is used by this equation is provided in Figure 1c.

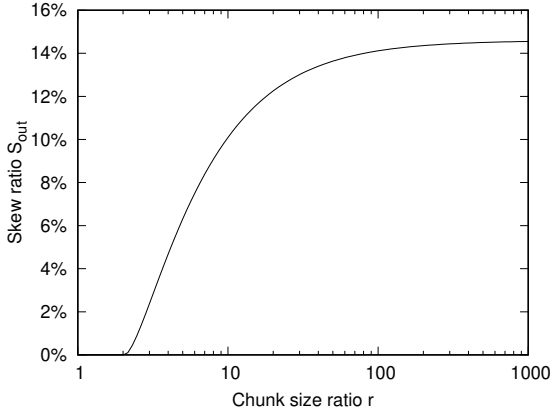
$$\begin{aligned} \mu_{out} &= \sum_{i=MinSize}^{MaxSize} i \cdot P(X = i) \\ &= \frac{1}{2}(MinSize + MaxSize)(1 + S_{out}) \quad (2) \\ S_{out} &\approx \begin{cases} 0 & 1 \leq r < 2 \\ \frac{0.585 - 0.585r + 0.146r^2}{-0.979 - 0.654r + r^2} & 2 \leq r \end{cases} \end{aligned}$$



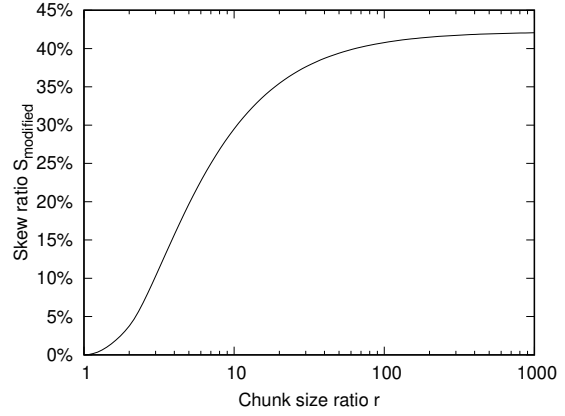
(a) Chunk size distribution.



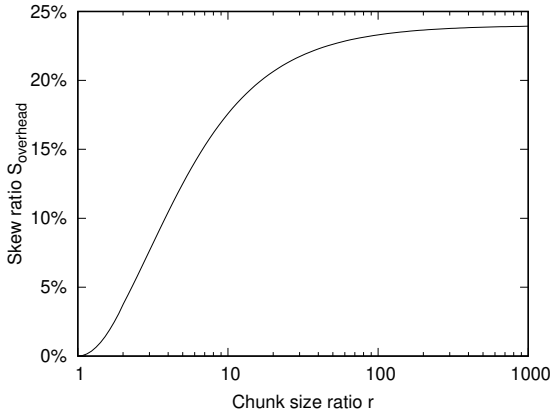
(b) Chunk size probability contrast.



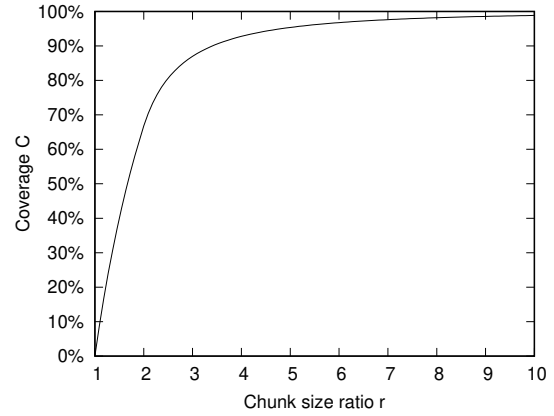
(c) Skew of mean size of chunks generated by the algorithm.



(d) Skew of the mean size of chunks as the result of single-byte modifications.



(e) Skew of the mean size of chunks as the result of single-byte modifications, relative to the mean size of chunks generated by the algorithm.



(f) Hash function coverage.

Figure 1: Behavior of MaxCDC.

3.3 Single-byte modifications

When chunks generated by MaxCDC are stored in a CAS, the deduplicated set of objects will have an average size that is larger than μ_{out} . This is caused by the fact that larger chunks have a lower probability of getting deduplicated against existing objects than smaller ones.

It is hard to make general predictions of how much larger the average size of deduplicated objects is with respect to μ_{out} , as it depends on many factors, including the amount of redundancy present in the original data set. However, a simple case that we may analyze is the following. Assume a randomly selected byte in a file is modified, what is the additional amount of space that is needed to store the modified file in the CAS? Assuming such a change does not alter any cutting points (which is often the case), the additional space that is needed is equal to the size of the chunk containing the randomly selected byte. We define $\mu_{modified}$ as the mean size of the containing chunk, for which an approximation is provided in Equation 3. Figure 1d is graph of skew factor $S_{modified}$ that is used by this approximation.

$$\begin{aligned}\mu_{modified} &= \sum_{i=MinSize}^{MaxSize} \frac{i^2 \cdot P(X=i)}{\mu_{out}} \\ &= \frac{1}{2}(MinSize + MaxSize)(1 + S_{modified}) \\ S_{modified} &\approx \begin{cases} \frac{r^2-2r+1}{3(r+1)^2} & 1 \leq r < 2 \\ \frac{1.131-1.343r+0.422r^2}{-0.909+0.225r+r^2} & 2 \leq r \end{cases}\end{aligned}\quad (3)$$

It is of particular interest to determine how much larger chunks caused by single-byte modifications are with respect to those generated by the algorithm originally, as this corresponds to the overhead of making single-byte modifications. This is expressed by skew factor $S_{overhead}$. An approximation of this skew factor is given in Equation 4, corresponding to the graph in Figure 1e.

$$\begin{aligned}S_{overhead} &= \frac{\mu_{modified}}{\mu_{out}} - 1 \\ &\approx \begin{cases} \frac{r^2-2r+1}{3(r+1)^2} & 1 \leq r < 2 \\ \frac{-0.117-0.201r+0.240r^2}{3.626+2.073r+r^2} & 2 \leq r \end{cases}\end{aligned}\quad (4)$$

From this graph we can conclude that using a smaller chunk size ratio may be advantageous, as it leads to less storage space usage in case of single-byte

modifications. This trivially makes sense, as using fixed-size chunks (i.e., $r = 1$) would behave optimally in this regard.

3.4 Coverage

Another aspect that is useful to consider is how the value of r influences the algorithm's ability to continue to find and select the most desirable cutting points, even when data shifts due to insertions and removals.

Whenever MaxCDC makes a cut at offset o close to $MaxSize$, i.e. $o > MaxSize - MinSize$, it will introduce a region between offsets $MaxSize$ and $o + MinSize$ for which the algorithm never computes rolling hashes, similar to sub-minimum chunk cut-point skipping.

We define the coverage C of the algorithm as the ratio of how many points in range of the input file in which cuts are allowed to be made, are actually considered for cutting. This can be interpreted as the probability at which the algorithm is capable of making a cut at the most desirable point in the file at which the rolling hash value is maximal. An approximation for this ratio is given in Equation 5, under the assumption the chunk size distribution is the same as simulated previously. A graph is provided in Figure 1f.

$$\begin{aligned}c(s) &= \min(s, MaxSize - MinSize + 1) \\ \mu_{covered} &= \sum_{i=MinSize}^{MaxSize} c(i) \cdot P(X=i) \\ C &= \frac{\mu_{covered}}{\mu_{out}} \\ &\approx \begin{cases} \frac{2(r-1)}{r+1} & 1 \leq r < 2 \\ \frac{-5.210+2.194r+r^2}{-3.576+2.166r+r^2} & 2 \leq r \end{cases}\end{aligned}\quad (5)$$

The key takeaway is that using MaxCDC requires making a trade-off. Setting chunk size ratio r low causes all chunks to be similar in size, which makes resource utilization of requests against a CAS predictable. It also minimizes the size of chunks resulting from single-byte modifications. However, it does prevent the algorithm from selecting the most desirable cutting points due to coverage being low. In Section 6.3 we will measure the optimal value of r for some data sets.

4 RecMaxCDC: divide-and-conquer chunking

What prevents MaxCDC from always cutting at the most desirable points is that the algorithm selects cutting points eagerly using a simple forward scan. Let's address this by considering an algorithm that uses a holistic approach. Namely, one that initially scans all eligible points in the input file to find the most desirable cutting point. After making a cut at this point, both halves are chunked by applying the same process recursively. For sufficiently large input files, this process always yields chunks with size $[MinSize, 2 \cdot MinSize)$. Pseudocode for this algorithm, which we'll call RecMaxCDC (*recursive MaxCDC*), is provided in Algorithm 4.

Algorithm 4 RecMaxCDC

```

procedure GETALLCUTS(data)
    return GETCUTSINRANGE(data, 0, |data|)
end procedure

procedure GETCUTSINRANGE(data,  $i^{\triangleleft}$ ,  $i^{\triangleright}$ )
    if  $i^{\triangleright} - i^{\triangleleft} < 2 \cdot MinSize$  then
        // Range is too small to chunk.
        return  $\emptyset$ 
    end if

    // Divide and conquer.
     $i^* \leftarrow \text{FINDCUT}(\textit{data}, i^{\triangleleft}, i^{\triangleright})$ 
     $C^{\triangleleft} \leftarrow \text{GETCUTSINRANGE}(\textit{data}, i^{\triangleleft}, i^*)$ 
     $C^{\triangleright} \leftarrow \text{GETCUTSINRANGE}(\textit{data}, i^*, i^{\triangleright})$ 
    return  $C^{\triangleleft} \cup \{i^*\} \cup C^{\triangleright}$ 
end procedure

procedure FINDCUT(data,  $i^{\triangleleft}$ ,  $i^{\triangleright}$ )
    // Find the most desirable cutting point.
     $i^* \leftarrow i^{\triangleleft} + MinSize$ 
     $h^* \leftarrow \text{Hash}(\textit{data}[i^* - WindowSize : i^*])$ 
    for  $i \leftarrow i^* + 1$  to  $i^{\triangleright} - MinSize$  do
         $h \leftarrow \text{Hash}(\textit{data}[i - WindowSize : i])$ 
        if  $h^* < h$  then
            // Found a more desirable cutting point.
             $h^* \leftarrow h$ 
             $i^* \leftarrow i$ 
        end if
    end for
    return  $i^*$ 
end procedure

```

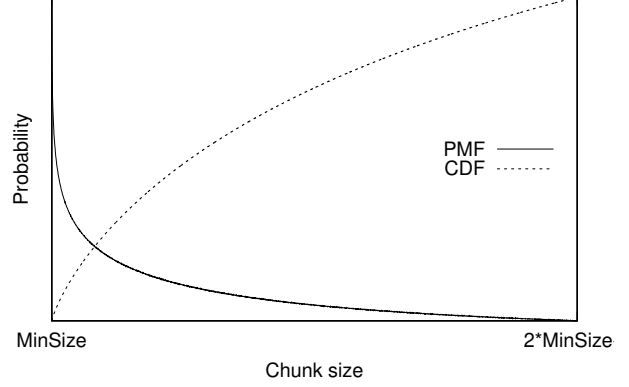


Figure 2: RecMaxCDC chunk size distribution.

4.1 Simulation

Algorithm 5 can be used to produce random numbers whose values correspond to the sizes of chunks produced by the RecMaxCDC algorithm when applied to random data. Using this algorithm, we were able to redo the same simulations as we performed for MaxCDC. By generating the sizes of 10^{10} chunks, we obtained the chunk size distribution shown in Figure 2. Based on this distribution, we were able to derive the values in Equation 6.

Algorithm 5 RecMaxCDC random chunk sizes

```

procedure GENERATERANDOMCHUNKSIZES
     $R = []$ 
    loop
        // Initially create a range with random size.
        if  $|R| = 0$  then
             $R \leftarrow R \parallel [\text{rand}()]$ 
        end if

         $e = R.\text{pop}()$ 
        if  $e < MinSize$  then
            // Range is small enough. Yield it.
            yield  $MinSize + e$ 
        else
            // Range is too large. Cut it.
             $t = e - MinSize$ 
             $c = \text{rand}() \bmod (t + 1)$ 
             $R \leftarrow R \parallel [c, t - c]$ 
        end if
    end loop
end procedure

```

$$\begin{aligned}
\mu_{out} &\approx 1.3376 \cdot \text{MinSize} \\
\mu_{modified} &\approx 1.3970 \cdot \text{MinSize} \\
S_{overhead} &\approx 4.44\% \\
C &= 100\%
\end{aligned} \tag{6}$$

The expected overhead of single-byte modifications against randomly generated files $S_{overhead}$ is equal to that of MaxCDC with chunk size ratio $r \approx 2.132$. While RecMaxCDC provides full coverage, MaxCDC would at this chunk size ratio only have coverage $C \approx 71.7\%$.

Of particular interest is the constant 1.3376 that shows up in the approximation of μ_{out} . Its reciprocal, 0.74759..., appears in Rényi's parking problem [9] and is often referred to as Rényi's parking constant. There it represents the density at which unit-sized cars may be parked randomly along a road, as the length of the road goes to infinity. Its appearance is not a coincidence, as for random input data the selection of cutting points as performed by RecMaxCDC is merely a discretized version of Rényi's parking problem. This version of Rényi's parking problem has been studied by Clay and Simányi [2].

4.2 Targeted searches for cutting points

Algorithm 4 succinctly captures how RecMaxCDC works, but it is by no means intended to describe what an efficient implementation looks like. As FINDCUT searches for the point where the rolling hash value is maximal within the entire range, data is hashed redundantly during recursion. There is also no guarantee that points at which cuts are made are close to the center of the range. This leads to a worst-case running time is quadratic, similar to Quicksort when pivots are selected poorly.

An observation we can make is that the divide-and-conquer strategy used by GETCUTSINRANGE does not depend on FINDCUT actually selecting the most desirable cutting point in the provided range. The algorithm will continue to produce the same results, even if the order in which FINDCUT selects points differs.

We may therefore design an improved version of FINDCUT that performs a targeted search for cutting points close to the center of the range, as demonstrated in Algorithm 6. This algorithm makes use of the property that any point at which the rolling hash value is a *local maximum* is part of the result

set. In this context a local maximum means that the rolling hash value is higher than at any of the $\text{MinSize} - 1$ points before it, and not lower than at any of the $\text{MinSize} - 1$ points after it. As this is only a sufficient and not a necessary condition, searching for such points may not always succeed (e.g., in case all bytes in the scanned region have the same value). This requires the algorithm to fall back to calling the original FINDCUT function if needed.

Algorithm 6 Improved version of FINDCUT that attempts to cut close to the center

```

procedure FINDCUTV2( $data, i^{\triangleleft}, i^{\triangleright}$ )
  if  $i^{\triangleright} - i^{\triangleleft} \geq 10 \cdot \text{MinSize}$  then
    // Start searching in the center.
     $i^* \leftarrow \lfloor \frac{i^{\triangleleft} + i^{\triangleright}}{2} \rfloor$ 
     $h^* \leftarrow \text{Hash}(data[i^* - \text{WindowSize} : i^*])$ 
     $i^+ \leftarrow i^* + \text{MinSize} - 1$ 
    for  $i \leftarrow i^* + 1$  to  $i^{\triangleright} - \text{MinSize}$  do
      if  $i^* \geq i^+$  and  $i = i^* + \text{MinSize}$  then
        //  $i^*$  is a local maximum.
        return  $i^*$ 
      end if

     $h \leftarrow \text{Hash}(data[i - \text{WindowSize} : i])$ 
    if  $h^* < h$  then
      // More desirable cutting point.
       $i^* \leftarrow i$ 
       $h^* \leftarrow h$ 
    end if
  end for

  if  $i^* \geq i^+$  then
    // Found a cutting point near the end.
    return  $i^*$ 
  end if
  end if

  // Fall back to scanning the full range.
  return FINDCUT( $data, i^{\triangleleft}, i^{\triangleright}$ )
end procedure

```

Algorithm 6 may be generalized to cut a large input file into an arbitrary number of roughly equally sized pieces. which can be achieved by repeatedly running this function with different initial values for i^* . This may be used as a preprocessing step to perform chunking of large files in parallel.

Adding parallelism this way differs from the techniques for BSW CDC functions that were presented in Section 2.2, as those are designed to be postpro-

cessing steps. However, finding cutting points this way does share similarities with MUCH’s approach of finding marker chunks.

4.3 Sequential algorithm

Searching for points at which the rolling hash value is a local maximum can also act as the basis for a fully sequential version of RecMaxCDC, as shown in Algorithm 7. In order to report all cutting points and not just the ones corresponding to local maxima, this algorithm stores all intermediate points at which a new maximum rolling hash value is observed in list I . Whenever it is determined that the last inserted point corresponds to a local maximum, a reverse pass against list I is made to cut at all points that are at least $MinSize$ bytes apart. This approach allows the algorithm to report all cutting points without hashing any data redundantly, thereby making it run in linear time.

Algorithm 7 has a worst-case memory usage that is linear with respect to the size of the input file. Namely, if an input file is structured in such a way that all newly observed maximum rolling hash values are less than $MinSize$ bytes apart, then the algorithm is only capable of calling PICKMAXIMUMCUTS and clearing list I when reaching the end-of-file. However, as the value of h^* increases, it should become less likely that new potential cutting point are encountered. The distance between the points in list I is therefore expected to increase, before ultimately finding a point at which the rolling hash value is a local maximum.

As it may be of interest to either preallocate list I or model it as a fixed-size list, we performed measurements of how large this list may become when chunking realistic data sets. The results are provided in Section 6.5. In order to protect against maliciously crafted files intentionally exhausting a fixed-size list, one may use a rolling hash function that depends on a secret key (KCDC). [10]

5 RepMaxCDC: restoring limited lookahead

Even though Algorithm 7 processes input data in a streaming manner, it places no bound on the distance between hashing of data and reporting of cutting points. This may prevent RecMaxCDC from being used as a drop-in replacement for conventional CDC algorithms in settings where streaming data needs to

Algorithm 7 Sequential version of RecMaxCDC

```

procedure GETALLCUTS( $data$ )
   $n \leftarrow |data|$ 
  if  $n < 2 \cdot MinSize$  then
    // File is too small to chunk.
    return  $\emptyset$ 
  end if

   $I \leftarrow [MinSize]$ 
   $h^* \leftarrow Hash(data[I_{|I|} - WindowSize : I_{|I|}])$ 
  for  $i \leftarrow I_{|I|} + 1$  to  $n - MinSize$  do
     $h \leftarrow Hash(data[i - WindowSize : i])$ 
    if  $i = I_{|I|} + MinSize$  then
      //  $I_{|I|}$  is a local maximum.
       $h^* \leftarrow h$ 
      PICKMAXIMUMCUTS( $I$ )
       $I \leftarrow [i]$ 
    else if  $h^* < h$  then
      // Found a more desirable cutting point.
       $h^* \leftarrow h$ 
       $I \leftarrow I || [i]$ 
    end if
  end for

  // Complete lingering points.
  PICKMAXIMUMCUTS( $I$ )
end procedure

procedure PICKMAXIMUMCUTS( $I$ )
   $m \leftarrow \infty$ 
  for  $n \leftarrow |I|$  downto 1 do
    if  $I_n \leq m - MinSize$  then
      // Report the cutting point.
      yield  $I_n$ 
       $m \leftarrow I_n$ 
    end if
  end for
end procedure

```

be chunked, and subsequently processed and stored, all using a fixed-size read buffer.

To address this, we present Algorithm 8. This algorithm is identical to plain MaxCDC, except that it is applied *repeatedly* until the resulting chunk is smaller than $2 \cdot \text{MinSize}$, hence the name RepMaxCDC.

Algorithm 8 RepMaxCDC

```

procedure GETFIRSTCHUNKLENGTH(data)
   $n \leftarrow \min(|\text{data}|, 2 \cdot \text{MinSize} + \text{HorizonSize})$ 
  while  $n \geq 2 \cdot \text{MinSize}$  do
     $i^* \leftarrow \text{MinSize}$ 
     $h^* \leftarrow \text{Hash}(\text{data}[i^* - \text{WindowSize} : i^*])$ 
    for  $i \leftarrow i^* + 1$  to  $n - \text{MinSize}$  do
       $h \leftarrow \text{Hash}(\text{data}[i - \text{WindowSize} : i])$ 
      if  $h^* < h$  then
         $h^* \leftarrow h$ 
         $i^* \leftarrow i$ 
      end if
    end for
     $n \leftarrow i^*$ 
  end while
  return  $n$ 
end procedure

```

HorizonSize controls the amount of lookahead used by RepMaxCDC. There are certain ranges of values that are particularly noteworthy:

- *HorizonSize* = 0: RepMaxCDC generates fixed-size chunks.
- *HorizonSize* < *MinSize*: RepMaxCDC behaves identically to MaxCDC with *MaxSize* = *MinSize* + *HorizonSize*.
- *HorizonSize* $\geq 2 \cdot (\text{MinSize} - 1)$: RepMaxCDC always cuts at points where the rolling hash value is a local maximum. This allows the techniques for chunking large files in parallel discussed in Section 4.2 to be reused.
- *HorizonSize* = ∞ : RepMaxCDC’s behavior is fully identical to RecMaxCDC.

Just like the basic algorithms presented in the previous sections, Algorithm 8 is not an efficient implementation of RepMaxCDC. Data is hashed redundantly both within a single call and across calls to GETFIRSTCHUNKLENGTH.

A more efficient implementation can be obtained by extending RecMaxCDC’s Algorithm 7. Normally its loop body appends potential cutting points to list

I until a point at which the rolling hash value is a local maximum is encountered. At that point list *I* is fully consumed and reinitialized.

In the case of RepMaxCDC the loop body also needs to consider the case where no local maximum is found within the horizon (i.e., $i > I_0 + \text{HorizonSize}$). When this occurs, only a single cutting point needs to be extracted from list *I*. Algorithm 9 selects such a cutting point in a way that is consistent with Algorithm 8.

Algorithm 9 Forced cutting point selection

```

procedure FORCEPICKFIRSTCUT(I)
   $m \leftarrow \infty$ 
  for  $n \leftarrow |I|$  downto 1 do
    if  $I_n \leq m - \text{MinSize}$  then
       $m \leftarrow I_n$ 
    end if
  end for
  return  $m$ 
end procedure

```

Afterwards, list *I* needs to be readjusted to satisfy the invariant that I_0 always refers to point $k = m + \text{MinSize}$, where *m* is the point at which the last cut is made. Any points at the start of the list below *k* are no longer eligible and may thus be removed. If after removal $I_0 > k$, then any potential cutting points between $[k, I_0]$ need to be recomputed.

Recomputing these cutting points means that the algorithm no longer processes data in a fully sequential manner, as data needs to be hashed up to two times. In Section 6.4 we show that for a sufficiently large horizon size this code path is rarely exercised, meaning it does not affect performance measurably.

6 Benchmarks

In order to determine the efficacy of the novel algorithms, we perform tests using processes similar to those of Gregoriadis *et al.* [5]. We make use of the cdc-bench testing framework¹, commit **fce34df9** (2026-06-09). Data sets provided by this version of cdc-bench that we will consider are as follows:

- **CODE**: 244 uncompressed tarballs of source code distributions of various releases of open-source projects Emacs, GCC, GDB, and the Linux kernel. This data set is 115 GB in size.

¹github.com/mrd0ll4r/cdc-bench

- **LNx**: 54 ISO images of various versions of Arch Linux, Debian, Fedora, and Ubuntu. This data set is 116 GB in size.
- **WEB**: Tarballs of daily snapshots of the website nytimes.com for January 2024, downloaded from the Internet Archive and recursively crawling up to a single level of links. This data set is 61 GB in size.

Additionally, we use the following data set that was compiled by Udayashankar *et al.* [11]:

- **DEB**: 62 Debian virtual machine images obtained from VMware Marketplace. This data set is 39 GB in size.

6.1 RecMaxCDC versus existing BSW algorithms

Of the newly presented algorithms, RecMaxCDC has the fewest configurable parameters. This makes it the most suitable to act as a baseline in our tests. We therefore start by comparing the deduplication ratio of RecMaxCDC against some of the Basic Sliding Window (BSW) CDC functions implemented by cdc-bench’s `cdc-algorithm-tester` utility. CDC functions that we will consider are as follows:

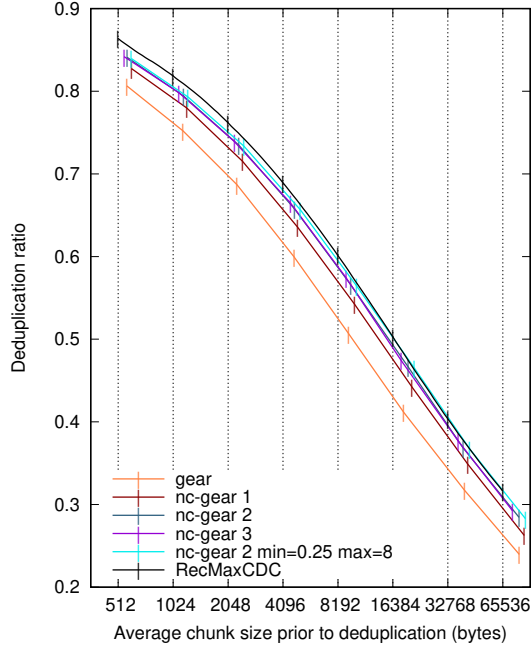
- **gear**: Plain BSW CDC function as described in Section 2 without having any chunk size bounds.
- **nc-gear 1**, **nc-gear 2**, and **nc-gear 3**: BSW CDC function that uses increasing levels of Normalized Chunking. The numerical argument corresponds to the additional number of 1-bits in *Mask* to set before, and to clear after reaching the target chunk size.
- **nc-gear 2 min=0.25 max=8**: BSW CDC function that uses Normalized Chunking at level 2. Additionally, it uses chunk size bounds $MinSize = 0.25 \cdot NormalSize$ and $MaxSize = 8 \cdot NormalSize$. This corresponds to typical use of FastCDC.

For consistency with the functions above, the implementation of RecMaxCDC also uses the Gear rolling hash function. Because RecMaxCDC compares rolling hash values for inequality, the top bits of these values play a more important role than the bottom bits. In order to compare the algorithms fairly, the *Mask* used by the BSW CDC functions will also have just the top bits set.

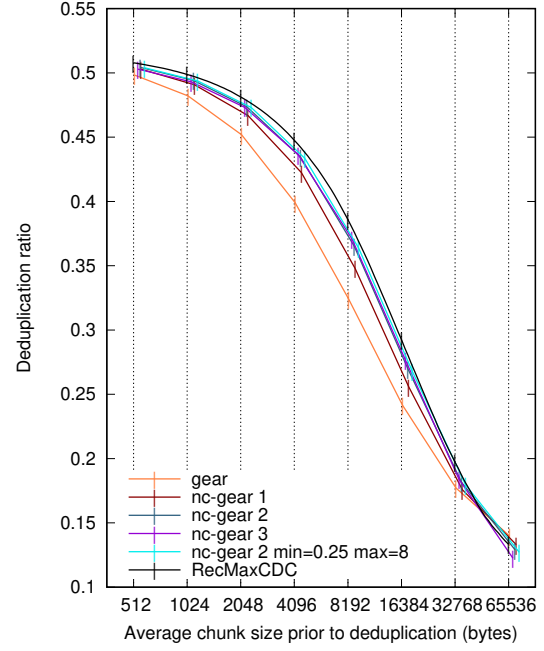
As the number of 1-bits in *Mask* determines the exponent of the target chunk size, we invoke the BSW CDC functions with target chunk sizes at powers of two between 512 and 65536 bytes. To let RecMaxCDC use the same target chunk sizes, we may invoke it with $MinSize = 512 \cdot 0.74759 \dots \cdot 2^i$. However, to get more insight in the smoothness of the deduplication ratio with respect to the target chunk size, we let RecMaxCDC use steps of $\sqrt[16]{2}$ instead.

In Figure 3 we provide graphs of the deduplication ratios of these algorithms when run against the aforementioned data sets. In these graphs the X axis represents the actually observed average chunk size prior to deduplication, which may differ from the configured target chunk size. Every sample corresponding a target chunk size of a power of two is marked with a vertical line. From these graphs we can conclude the following:

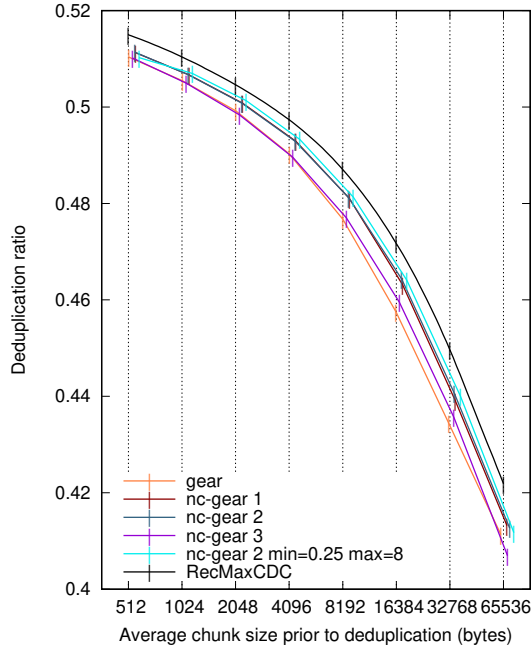
- For the vast majority of chunk sizes and data sets that were tested, RecMaxCDC provides a higher deduplication ratio than the BSW CDC functions.
- Consistent with previous results, we observe that of Normalized Chunking leads to better deduplication. However, for the **LNx** data set we observe that very high amounts of normalization leads to worse results.
- For RecMaxCDC, we observe that all marked samples are perfectly aligned with the grid lines of the X axis. This suggests that the average chunk size can be predicted with high accuracy using Rényi’s parking constant, even for non-random data. This is in contrast to the BSW CDC functions, where the average chunk size drifts above or below the target chunk size depending on the normalization level and data set at hand.
- In the **WEB** data set, we see RecMaxCDC’s deduplication ratio drop sharply at around 685 bytes. By inspecting the resulting chunks, we observe that this is caused by the minimum chunk size getting pushed beyond $685 \cdot 0.74759 \dots \approx 512$ bytes. As uncompressed tarballs have all records aligned at 512 bytes, using a minimum chunk size of 513 or above prevents RecMaxCDC from placing small records in their own chunk.



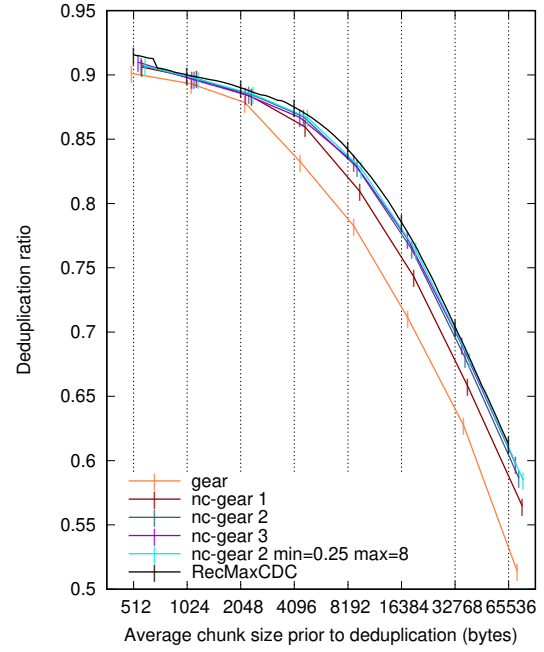
(a) **CODE**



(b) **DEB**



(c) **LNX**



(d) **WEB**

Figure 3: Deduplication ratios of RepMaxCDC and Basic Sliding Window functions.

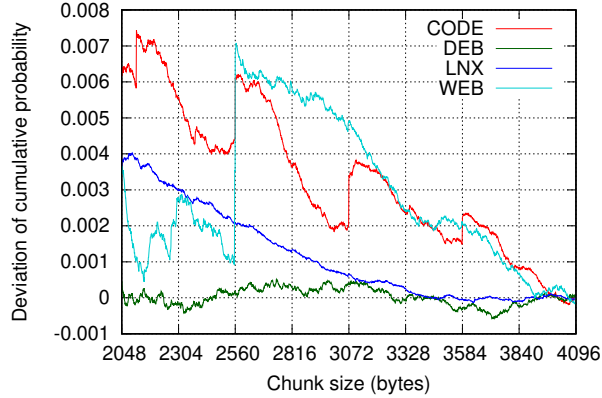


Figure 4: Deviation of the cumulative chunk size distribution of RecMaxCDC against data sets under test, relative to the simulated results in Section 4.1.

6.2 RecMaxCDC’s chunk size distribution

In the previous section we observed that RecMaxCDC’s average chunk size could be accurately predicted for the data sets under test using Rényi’s parking constant. This raises the question how closely the size distribution of chunks generated by RecMaxCDC matches that of the distance between cars in Rényi’s parking problem, which was simulated in Section 4.1.

To answer this question, we provide a graph in Figure 4 of the cumulative distribution of chunk sizes generated by RecMaxCDC, from which we subtract the distribution generated by Algorithm 5, i.e., $F_{actual}(x) - F_{simulated}(x)$. For both distributions we assume a minimum chunk size of 2048 bytes. From this graph we can conclude the following:

- The chunk size distributions of the data sets under test matches the simulated distribution quite closely, with the maximum deviation (i.e., the Kolmogorov–Smirnov statistic) being less than 0.8%.
- For most data sets we observe that the deviation is positive, meaning that RecMaxCDC creates chunks that are slightly smaller than what was simulated for random inputs. This can be explained by the fact that in case cuts are made inside regions containing repetitions, RecMaxCDC prefers cutting at the first occurrence.
- For the **LNX** data set we see a lower deviation than for **CODE** and **WEB**. This is a result of the ISO images in this data set contain-

ing compressed data, such as **.deb** and **.rpm** files. These typically contain less repetition than uncompressed files. As the **DEB** data set consists of compressed data almost exclusively, its chunk size distribution is indistinguishable from that of random data.

- For the **CODE** and **WEB** data sets we observe sharp rises at multiples of 512 bytes, which similar to what we saw in the previous section is caused by records in uncompressed tarballs that are aligned at 512 bytes.

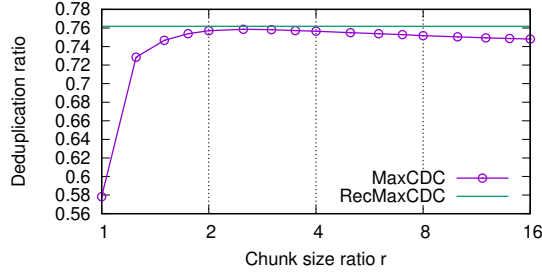
6.3 MaxCDC’s optimal chunk size ratio

In Section 3.4 we concluded that the use of MaxCDC requires making a trade-off. Setting the chunk size ratio r to a lower value results in less overhead for single-byte modifications (i.e., lower $S_{overhead}$), but setting it to a larger value provides better coverage of the input data (i.e., higher C).

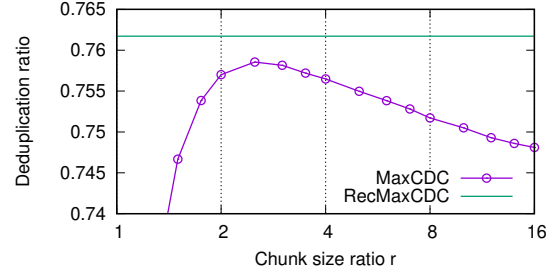
In this section we investigate whether this effect can actually be observed in practice. This is done by invoking MaxCDC with chunk size ratios varying between 1 and 16. For each chunk size ratio, we repeatedly adjust *MinSize* and *MaxSize*, so that the observed average chunk size before deduplication is as close to 2048 bytes as possible.

Graphs of the deduplication ratios of these runs are provided in Figure 5. For comparison, these graphs also include the deduplication ratio of RecMaxCDC for the same average chunk size. Close-ups are provided to better distinguish deduplication ratios for $r \geq 2$. From these graphs we can conclude the following:

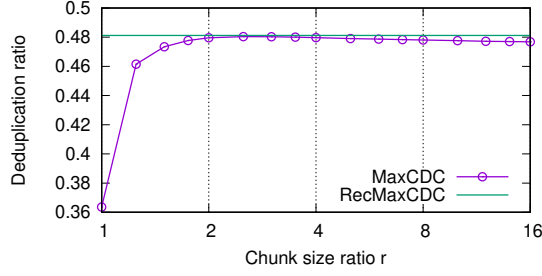
- The trade-off that was predicted can also be observed in practice. For all data sets we observe that choosing r either too low or too high results in worse deduplication ratios.
- The optimal value of r highly depends on the data set at hand. For the **CODE** and **DEB** data sets we achieve the highest amount of deduplication at $r = 2.5$, while for the **LNX** and **WEB** data sets it sits at $r = 4$ and $r = 6$, respectively. This means manual tuning is required to achieve optimal deduplication for this algorithm.
- With an optimal value of $r = 6$ or lower, the optimal spread in chunk size is significantly lower than that of conventional BSW CDC functions.



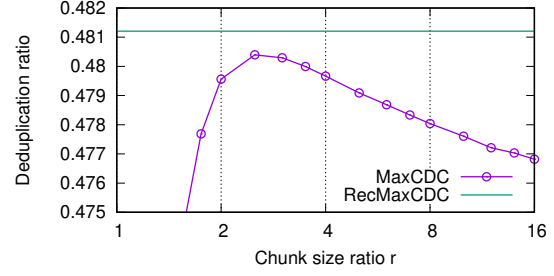
(a) **CODE**



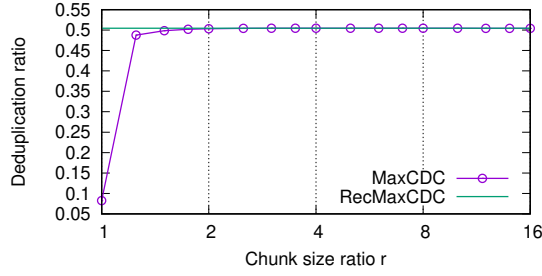
(b) **CODE, close-up**



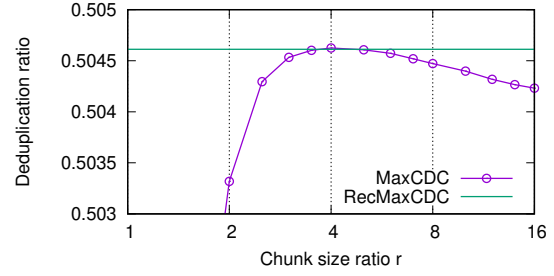
(c) **DEB**



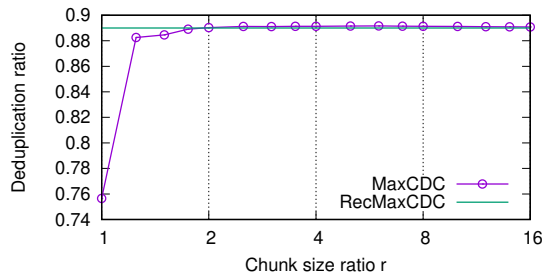
(d) **DEB, close-up**



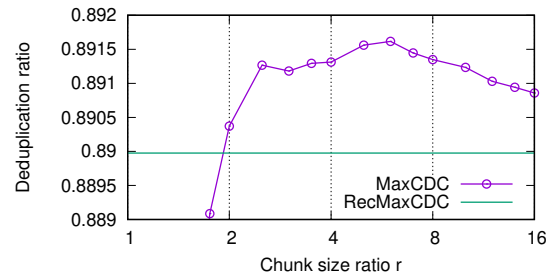
(e) **LNX**



(f) **LNX, close-up**



(g) **WEB**



(h) **WEB, close-up**

Figure 5: Deduplication ratios of MaxCDC with different chunk size ratios, given an average chunk size of 2048 bytes.

- For the **CODE** and **DEB** data sets we see that RecMaxCDC outperforms MaxCDC. For the **LNK** data set, MaxCDC is only able to keep up if r is chosen optimally. As in this case $r \geq 2$, RecMaxCDC achieves the same results with a smaller chunk size spread.
- For the **WEB** data set we see that MaxCDC outperforms RecMaxCDC slightly. As we established previously, this data set is strongly oriented around 512-byte tarball entries. With a target chunk size of 2048 bytes, RecMaxCDC is never capable of storing fewer than 3 of such entries in a single chunk. MaxCDC has an advantage here, as at $r \geq \frac{2}{0.74759...} - 1 \approx 1.675$, its minimum chunk size is smaller than that of RecMaxCDC.

6.4 RepMaxCDC's horizon size

RepMaxCDC's *HorizonSize* limits the amount of lookahead when searching for points in the input file at which the rolling hash value is a local maximum. Setting it to a larger value increases the probability of RepMaxCDC's output matching that of RecMaxCDC. However, doing so does require the use of a larger read buffer.

In order to establish what is a reasonable limit, we process the data sets using a version of RepMaxCDC that records and aggregates the offset in the horizon at which a local maximum is found. When using a minimum chunk size of 2048 bytes we obtain the geometric-like distributions shown in Figure 6. These show that for each increment of *MinSize* the probability of scanning that far in the horizon decreases roughly by a factor 10.

As simulations at different chunk sizes yield similar distributions, we may conclude that setting $\text{HorizonSize} = 8 \cdot \text{MinSize}$ is a good starting point.

6.5 RecMaxCDC's list I

The tests performed in the previous section were also used to measure the length of list I used by Algorithm 7 as it is passed to PICKMAXIMUMCUTS to select the definitive set of cutting points between local maxima. The distribution of the observed length is provided in Figure 7.

As can be seen in this graph, none of our benchmarks have been able to trigger any cases in which list I grew beyond 31 elements. Preallocating it to

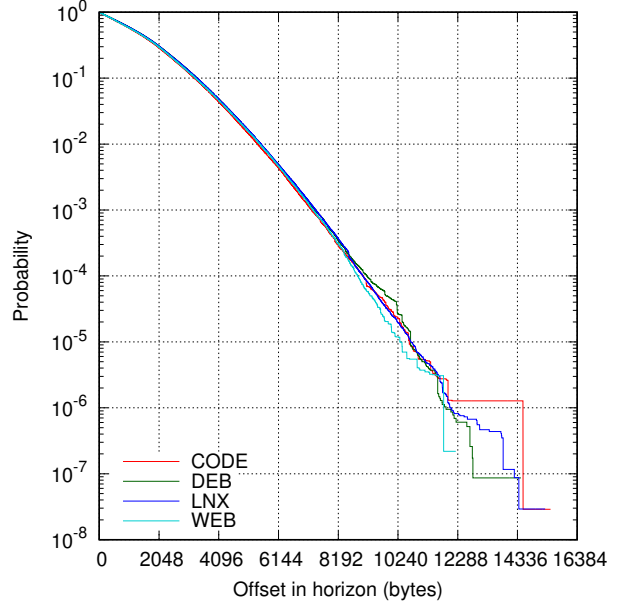


Figure 6: Probability of local maximum being located beyond an offset within the horizon, given a minimum chunk size of 2048 bytes.

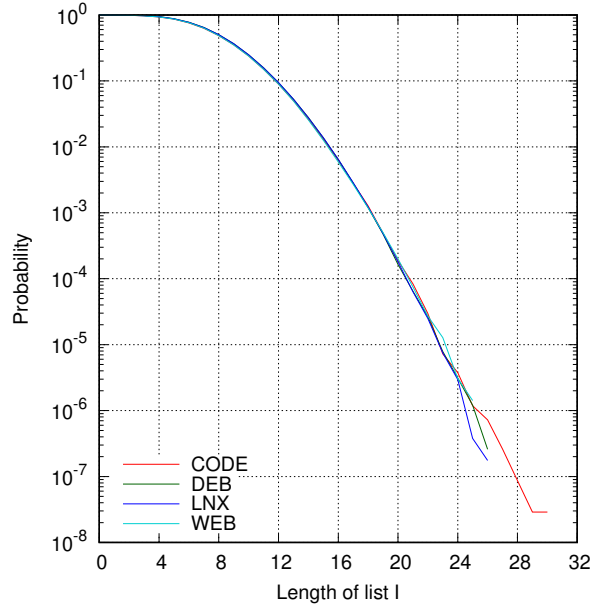


Figure 7: Probability of PICKMAXIMUMCUTS getting called with list I exceeding a certain length, given a minimum chunk size of 2048 bytes.

be 32 elements in size ought to be sufficient. For implementations that use a fixed-size list, a maximum of 64 elements would be a reasonable limit.

6.6 Throughput of RepMaxCDC

As stated previously, RepMaxCDC is designed to be an alternative to existing BSW CDC functions. Because its logic for selecting cutting points is more complex than that of traditional algorithms, it's worth investigating what impact this has on single-threaded throughput.

To measure this, implementations of RepMaxCDC and FastCDC were written in the Go programming language. Care was taken to make the implementations as similar as possible. For example, because the RepMaxCDC implementation has its Gear hashing loop unrolled to process eight bytes per iteration, the FastCDC implementation was written to do so as well. Both CDC functions are configured to use typical parameters:

- RepMaxCDC:
 - $HorizonSize = 8 \cdot MinSize$
- FastCDC:
 - $MinSize = 0.25 \cdot NormalSize$
 - $MaxSize = 8 \cdot NormalSize$
 - Normalized Chunking level 2

When building these implementations with Go compiler version 1.26.5 and running them on an Apple MacBook Pro with an M3 Pro CPU, we observe that chunking the **CODE** data set yields the throughput shown in Figure 8. From this graph we can conclude that RepMaxCDC is roughly 20% to 30% slower than FastCDC for the target chunk sizes and parameters that were tested.

The difference in speed can largely be attributed to FastCDC's use of sub-minimum chunk cut-point skipping (Section 2.1). As RepMaxCDC is explicitly designed to provide full coverage of input data, it is both infeasible and undesirable to apply such an optimization.

In Figure 9 we show the same results, except that this time throughput is measured in terms of data covered by the rolling hash function. This shows that when sub-minimum chunk cut-point skipping is disregarded, RepMaxCDC is less than 1.5% slower than FastCDC for a target chunk size of 8 KB and above.

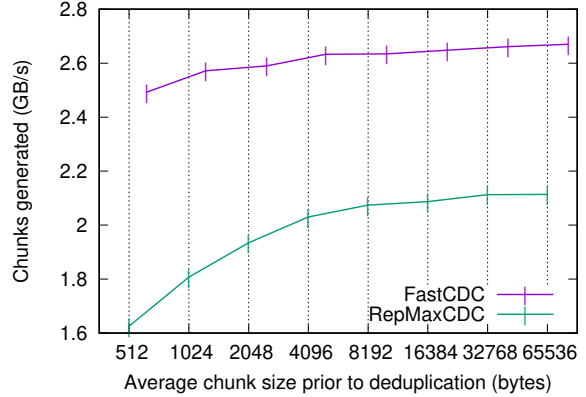


Figure 8: Rate at which chunks are generated when chunking the **CODE** data set, using various target chunk sizes.

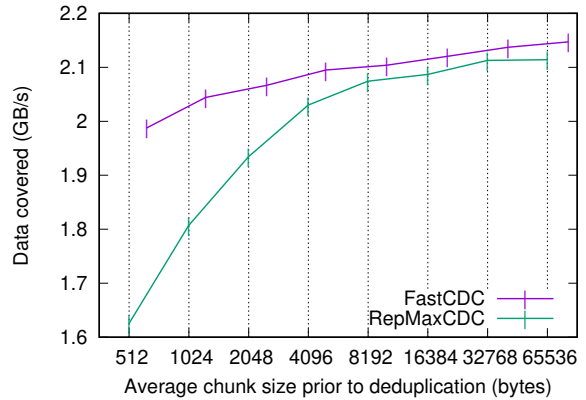


Figure 9: Rate at which data is covered by the hash function when chunking the **CODE** data set, using various target chunk sizes.

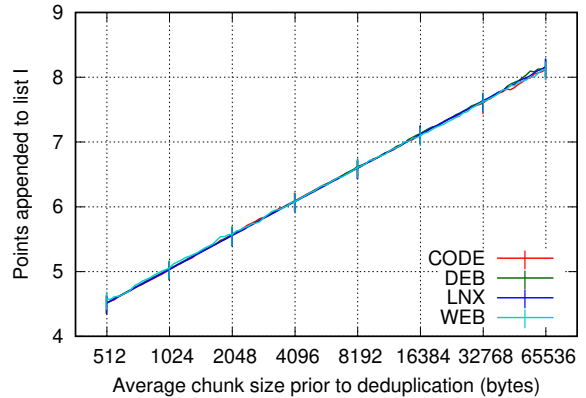


Figure 10: Average number of times points are appended to list *I* per chunk generated.

For small chunk sizes we see that RepMaxCDC’s throughput degrades more quickly than FastCDC’s. This can be attributed to the fact that for smaller chunk sizes, RepMaxCDC spends more time outside of its hot path where it computes and compares rolling hash values. In Figure 10 we show the average number of times points are appended to list I per chunk generated. As the number of append operations only grows logarithmically with respect to the average chunk size, selecting a larger target chunk size reduces the overall probability at which the hot path is left to insert points into list I .

7 Use and availability

MaxCDC was designed in March 2024 for use in Bonanza², a distributed build system that aims to be compatible with Bazel³. Bonanza’s storage subsystem only permits storing objects up to 2 MiB in size, as this ensures that requests against storage remain below gRPC’s 4 MiB message size limit. Using FastCDC with a maximum chunk size of 2 MiB would lead to a minimum chunk size of 64 KiB, which may under certain circumstances be too small for this system to work effectively.

In January 2026, Bazel’s remote execution protocol was extended to support CDC⁴. Given that this change had to preserve backward compatibility, this introduced certain ambiguities in the protocol where a single object identifier could refer to either an individual chunk of data or large files consisting of multiple chunks. This led to the design of the RepMaxCDC algorithm and subsequent inclusion into the protocol, as it eliminates this ambiguity by ensuring that sizes of chunks and large files don’t overlap. Bonanza followed suit by migrating away from MaxCDC to RepMaxCDC.

Reference implementations of MaxCDC and RepMaxCDC written in Go using the Gear rolling hash function are available on GitHub⁵, and are released under the Apache License 2.0. The same repository also contains tools to run some of the simulations and benchmarks performed in this paper.

In July 2026, the RepMaxCDC reference implementation was ported to Java and included into Bazel⁶.

²bonanza.build

³bazel.build

⁴github.com/bazelbuild/remote-apis/pull/357

⁵github.com/buildbarn/go-cdc

⁶github.com/bazelbuild/bazel/issues/30131

8 Conclusion

In this paper we presented a pair of CDC functions named RecMaxCDC and RepMaxCDC. These functions demonstrate that it is possible to achieve high rates of deduplication, even when bounds on chunk sizes are tight. Having only up to two configurable parameters (i.e., minimum chunk size and a horizon size), these algorithms are also easier to tune than some of the contemporary algorithms. The ability to parallelize the chunking process by performing targeted searches for cutting points is indispensable in the era of multiprocessor computing.

Content-Defined Chunking is used by secure backup services to achieve fast incremental backups. Truong [10] and Alexeev [1] have demonstrated that this use may be susceptible to fingerprinting attacks. An open question is whether RecMaxCDC’s tighter chunk size bounds make it less susceptible to such attacks, especially when combined with modern padding schemes like Padmé [7].

Prolly Trees are B-trees that are constructed using the same basic principles as Basic Sliding Window CDC functions⁷. These are used by version controlled databases like Noms and Dolt, as they yield the same tree representation regardless of the order in which writes occur. Because of the large spread in chunk size, Prolly Trees differ from traditional B-trees in that nodes can be less than 50% full. Changing them to use the same algorithm as RecMaxCDC may allow this property to be restored.

The relationship between RecMaxCDC and Rényi’s parking problem is most interesting. It may be worth investigating whether any past findings related to this problem can be translated to the domain of Content-Defined Chunking. It also raises the question whether the proposed algorithm is asymptotically optimal for any CDC function based on rolling hashes having such a tight chunk size bound.

9 Acknowledgments

The author would like to thank Marcel Gregoriadis and Leo Balduf for persuading him to write this paper.

The author would also like to use this opportunity to thank Benjamin Ingberg, Tyler French, and Ivan Futivić for their efforts getting RepMaxCDC adopted within the Bazel ecosystem.

⁷dolthub.com/docs/architecture/storage-engine/prolly-tree/

References

- [1] Boris Alexeev, Colin Percival, and Yan X Zhang. Chunking attacks on file backup services using content-defined chunking, 2025.
- [2] Matthew P. Clay and Nándor J. Simányi. Rényi’s parking problem revisited. *Stochastics and Dynamics*, 16(02):1660006, March 2016.
- [3] Jonathan D. Cohen. Recursive hashing functions for n-grams. *ACM Trans. Inf. Syst.*, 15(3):291–320, July 1997.
- [4] Kave Eshghi and Hsiu Khuern Tang. A framework for analyzing and improving content-based chunking algorithms, 2005.
- [5] Marcel Gregoriadis, Leonhard Balduf, Björn Scheuermann, and Johan Pouwelse. A thorough investigation of content-defined chunking algorithms for data deduplication, 2024.
- [6] Fan Ni, Xing Lin, and Song Jiang. Ss-cdc: A two-stage parallel content-defined chunking for deduplicating backup storage. In *Proceedings of the 12th ACM International Conference on Systems and Storage*, SYSTOR ’19, pages 86–96, New York, NY, USA, 2019. ACM.
- [7] Kirill Nikitin, Ludovic Barman, Wouter Lueks, Matthew Underwood, Jean-Pierre Hubaux, and Bryan Ford. Reducing metadata leakage from encrypted files and communication with purbs, 2019.
- [8] Michael O. Rabin. Fingerprinting by random polynomials, 1981.
- [9] Alfred Rényi. On a one-dimensional problem concerning random space-filling. *Publ. Math. Inst. Hung. Acad. Sci.*, 3:109–127, 1958.
- [10] Kien Tuong Truong, Simon-Philipp Merz, Matteo Scarlata, Felix Günther, and Kenneth G. Paterson. Breaking and fixing content-defined chunking. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, CCS ’25, page 2294–2308, New York, NY, USA, 2025. Association for Computing Machinery.
- [11] Sreeharsha Udayashankar, Abdelrahman Baba, and Samer Al-Kiswani. VectorCDC: Accelerating data deduplication with vector instructions. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 513–522, Santa Clara, CA, February 2025. USENIX Association.
- [12] Youjip Won, Kyeongyeol Lim, and Jaehong Min. Much: Multithreaded content-based file chunking. *IEEE Transactions on Computers*, 64(5):1375–1388, 2015.
- [13] Wen Xia, Hong Jiang, Dan Feng, Lei Tian, Min Fu, and Zhongtao Wang. P-dedupe: Exploiting parallelism in data deduplication system. In *2012 IEEE Seventh International Conference on Networking, Architecture, and Storage*, pages 338–347, 2012.
- [14] Wen Xia, Hong Jiang, Dan Feng, Lei Tian, Min Fu, and Yukun Zhou. Ddelta: A deduplication-inspired fast delta compression approach. *Performance Evaluation*, 79, September 2014.
- [15] Wen Xia, Yukun Zhou, Hong Jiang, Dan Feng, Yu Hua, Yuchong Hu, Qing Liu, and Yucheng Zhang. FastCDC: A fast and efficient Content-Defined chunking approach for data deduplication. In *2016 USENIX Annual Technical Conference (USENIX ATC 16)*, pages 101–114, Denver, CO, June 2016. USENIX Association.